Table of Contents

El código de XENTRA explicado desde cero

Para quién es: para quien nunca ha programado y quiere entender **qué hay dentro** de XENTRA, **cómo se construyó**, **por qué** se hizo así y **para qué sirve** cada pieza.

Cómo leerlo: en orden. Cada parte se apoya en la anterior. Los fragmentos de código son **reales**, copiados del proyecto: no hace falta entenderlos línea por línea, lea los comentarios en español que los acompañan.

Contenido

1. La idea general: XENTRA es un restaurante
2. Diez palabras que debe conocer
3. El mapa de carpetas
4. Por qué se eligió cada tecnología
5. La cocina (backend), capa por capa
6. El salón (frontend), capa por capa
7. El viaje completo de un clic: "Firmar la atención"
8. Dónde vive la seguridad en el código
9. La bodega (base de datos) y sus cambios (migraciones)
10. Docker: cómo se empaca y se arranca todo
11. Las pruebas automáticas
12. Cómo se construyó, en qué orden y por qué
13. Ejercicio: cómo se agregaría un campo nuevo
14. Qué leer primero si quiere aprender más

---

1. La idea general: XENTRA es un restaurante

Imagine un restaurante:

En el restaurante	En XENTRA	Tecnología	Carpeta
El salón y la carta: lo que ve el cliente	Las **pantallas** que ve el médico en el navegador	Vue + TypeScript	`frontend/`
El mesero: lleva pedidos del salón a la cocina	Las **peticiones** que viajan del navegador al servidor	HTTP / API	`frontend/src/api/`
La cocina: donde	El **servidor**, que	Python + FastAPI	`backend/`

En el restaurante	En XENTRA	Tecnología	Carpeta
se prepara todo siguiendo recetas	aplica las reglas (quién puede firmar, cómo se genera un RIPS…)		
La bodega: donde se guardan los ingredientes	La **base de datos**, donde se guardan pacientes, historias, facturas	PostgreSQL	(volumen de Docker)
El libro del notario: anota todo, no se puede borrar	El **ledger** (cadena tipo blockchain)	SHA-256 + HMAC	`backend/app/services/ledger.py`
El camión de comida: lleva el restaurante armado a cualquier parte	**Docker**: empaca todo para que funcione igual en cualquier computador	Docker	`docker-compose.yml`

Un ejemplo completo: el médico escribe una consulta y pulsa **Firmar**.

1. La **pantalla** (salón) toma lo que escribió.
2. El **mesero** (petición HTTP) lo lleva a la cocina.
3. La **cocina** (backend) revisa que sea médico, que haya diagnóstico, calcula la huella digital y la anota en el **libro del notario** (ledger).
4. Guarda todo en la **bodega** (base de datos).
5. Le responde al salón "listo", y la pantalla muestra el folio sellado.

¿Por qué separar salón, cocina y bodega? Por la misma razón que en un restaurante: **el cliente no entra a la cocina**. El navegador del usuario nunca toca la base de datos. Todo pasa por la cocina, que es donde están las reglas y los controles de seguridad.

---

2. Diez palabras que debe conocer

Palabra	Qué es, en simple
Código	Instrucciones escritas para que el computador las siga, como una receta
Lenguaje	El "idioma" de esas instrucciones. Aquí se usan **Python** (cocina), **TypeScript** (salón) y **SQL** (bodega)
Framework	Un kit con piezas ya hechas para no empezar de cero. **FastAPI** para la

Palabra	Qué es, en simple
	cocina y **Vue** para el salón
Servidor	El computador (o programa) que atiende peticiones. En su caso, su PC con Docker
API	La "carta" de la cocina: la lista de pedidos que se pueden hacer (ej. "crear paciente", "firmar atención")
Endpoint	Un plato de esa carta: una dirección concreta, ej. `POST /api/atenciones/{id}/firmar`
Base de datos	Un archivador gigante y ordenado en tablas (como hojas de Excel con reglas estrictas)
Hash	Una "huella digital" de un texto. Si cambia una coma, la huella cambia por completo
Contenedor	Una caja cerrada que trae un programa con todo lo que necesita para funcionar
Git / GitHub	Git guarda el historial de cambios del código (cada "commit" es una foto). GitHub es donde se guarda en internet

3. El mapa de carpetas

```
XENTRA-DATACLINIC/
│
├── backend/                  ← LA COCINA (Python)
│   ├── app/
│   │   ├── main.py           ← La puerta de la cocina: arranca todo y conecta las ventanillas
│   │   ├── core/             ← Las instalaciones: configuración, conexión a la BD, seguridad
│   │   ├── models.py         ← Los planos de la bodega: cómo es cada tabla (Paciente, Atención…)
│   │   ├── models_ext.py     ← Más planos: agenda, facturas, camas, farmacia, laboratorio…
│   │   ├── schemas.py        ← Los formularios: qué datos se aceptan y cómo se validan
│   │   ├── schemas_ext.py    ← Más formularios
│   │   ├── api/              ← Las ventanillas: una por área (pacientes, atenciones, farmacia…)
│   │   ├── services/         ← Las recetas: la lógica (RIPS, ledger, PDF, IA, FHIR, RDA…)
│   │   ├── seed.py           ← Datos iniciales: administrador, catálogos, usuarios demo
│   │   ├── crear_usuarios.py ← Herramienta: crear usuarios desde un CSV
│   │   └── restablecer_clave.py ← Herramienta: restablecer una clave
│   ├── alembic/versions/     ← Las remodelaciones de la bodega (migraciones 0001, 0002, 0003)
│   ├── tests/                ← Las pruebas automáticas (60 pruebas)
│   ├── requirements.txt      ← La lista de ingredientes (librerías de Python)
│   └── Dockerfile            ← Cómo se empaca la cocina en un contenedor
│
├── frontend/                 ← EL SALÓN (Vue + TypeScript)
│   ├── index.html            ← La única página HTML; Vue dibuja todo dentro de ella
│   ├── public/               ← Imágenes fijas: logo y favicon
│   ├── src/
│   │   ├── main.ts           ← Arranca la aplicación Vue
│   │   ├── App.vue           ← El marco: menú lateral + espacio para cada pantalla
│   │   ├── router/           ← El mapa: qué pantalla corresponde a cada dirección
│   │   ├── stores/auth.ts    ← La memoria de la sesión: quién está conectado
│   │   ├── api/              ← El mesero: cómo se habla con la cocina
```

```
│   │   ├── views/            ← Las pantallas (23): Pacientes, Atención, Facturación…
│   │   ├── components/       ← Piezas reutilizables: buscador CIE-10, firma, dictado, gráficos
│   │   └── style.css         ← Los colores y estilos
│   ├── nginx.conf            ← El portero: entrega las pantallas y pasa los pedidos a la cocina
│   └── Dockerfile            ← Cómo se empaca el salón
│
├── db/                       ← Scripts de la bodega: usuario sin privilegios y respaldos diarios
├── docker-compose.yml        ← La orquesta: arranca bodega, cocina, salón y respaldos juntos
├── docker-compose.prod.yml   ← Variante para internet con HTTPS (Caddy)
├── docker-compose.tunel.yml  ← Variante para publicar con Cloudflare Tunnel
├── docker-compose.vpn.yml    ← Variante HCE privada por VPN WireGuard
├── .env.example              ← Plantilla de configuración (claves, datos de la IPS)
├── .github/workflows/ci.yml  ← Robot que corre las pruebas en GitHub en cada cambio
└── docs/                     ← Documentación, manuales, protocolos, marca
```

Regla para recordar: `backend` es la cocina, `frontend` es el salón, `docs` son los manuales, y los `docker-compose` son las instrucciones para montar todo.

---

4. Por qué se eligió cada tecnología

Pieza	Elegida	Por qué	Alternativas que se descartaron
Lenguaje de la cocina	**Python**	Es el lenguaje más fácil de leer, el más usado en IA y datos, y en Colombia hay muchos desarrolladores que lo conocen	Java (más pesado y lento de escribir), PHP (menos adecuado para IA)
Framework de la cocina	**FastAPI**	Valida automáticamente los datos, genera sola la documentación de la API (`/docs`) y es muy rápido	Django (trae mucho que no se necesitaba), Flask (habría que armar a mano la validación)
Bodega	**PostgreSQL**	Gratuita, de código abierto y muy confiable. Permite **triggers**: reglas dentro de la propia base de datos que impiden alterar historias firmadas	MySQL (triggers más limitados), Oracle/SQL Server (licencias costosas)
Traductor cocina-bodega	**SQLAlchemy**	Permite escribir en Python en vez de en SQL, y	Escribir SQL a mano (más errores y riesgos)

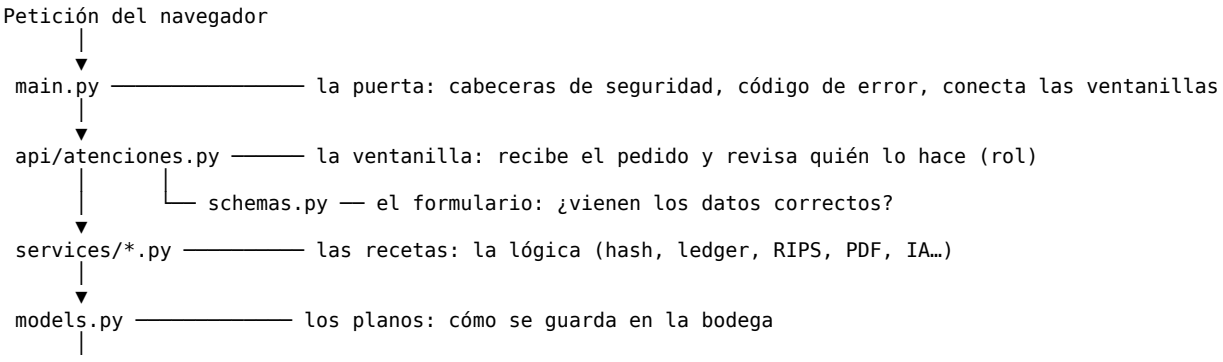
Pieza	Elegida	Por qué	Alternativas que se descartaron
		protege contra ataques de inyección SQL	
Salón	**Vue 3**	Es el más fácil de aprender de los frameworks modernos y produce pantallas rápidas	React (más difícil para principiantes), Angular (mucho más complejo)
Lenguaje del salón	**TypeScript**	Es JavaScript con "tipos": avisa los errores **antes** de que lleguen al usuario	JavaScript solo (los errores aparecen en producción)
Empaque	**Docker**	Funciona igual en su PC, en un servidor o en la nube. Se instala con un solo comando	Instalar todo a mano en cada equipo (lento y propenso a errores)
Cambios de la bodega	**Alembic**	Lleva un historial de cada cambio a las tablas, sin perder datos	Modificar tablas a mano (riesgo de perder información)

Criterio general: tecnologías **gratuitas, de código abierto, maduras y fáciles de conseguir quien las mantenga en Colombia**. Así la institución no queda amarrada a un proveedor ni a licencias.

---

5. La cocina (backend), capa por capa

La cocina está organizada como una cocina profesional: cada cosa en su estación.



```
                ▼
PostgreSQL ──────────── la bodega, con sus propias reglas (triggers)
```

5.1 `main.py`: la puerta

Es lo primero que corre. Hace tres cosas:

1. Al arrancar, prepara los datos iniciales (`seed.py`).
2. Pone **cabeceras de seguridad** a cada respuesta, por ejemplo que el navegador no guarde datos clínicos en caché.
3. **Conecta las ventanillas** (cada archivo de `api/`):

```
for modulo in (auth, pacientes, atenciones, cie10, rips, auditoria, administracion, publico, agenda,
facturacion,
               hospital, farmacia, laboratorio, fhir, portal, reportes, rda, bi, modulos, sistema):
    app.include_router(modulo.router)      # "abre" la ventanilla de cada área
```

Si algo falla, no muestra el error técnico al usuario. Le entrega un **código** para soporte:

```
return JSONResponse(status_code=500, content={
    "detail": f"Ocurrió un error inesperado. Reporte este código a soporte: {codigo}", ...})
```

¿Por qué? Los mensajes técnicos asustan al usuario y le dan pistas a un atacante. El código permite que soporte encuentre el error exacto en el registro.

5.2 `core/`: las instalaciones

Archivo	Para qué
`config.py`	Lee la configuración del archivo `.env` (claves, datos de la IPS, si el portal está activo…)
`database.py`	Abre la conexión con PostgreSQL
`security.py`	Cifra contraseñas (Argon2id) y crea los "pases de entrada" (tokens JWT)
`deps.py`	El **guardia**: verifica quién es el usuario y si su rol puede hacer lo que pide
`inmutabilidad.py`	Las reglas que se instalan **dentro** de PostgreSQL para impedir alterar lo firmado

El guardia es muy sencillo de leer:

```
def requiere_rol(*roles: str):
    """Uso: Depends(requiere_rol("medico", "admin"))"""
    def verificar(usuario: Usuario = Depends(usuario_actual)) -> Usuario:
        if usuario.rol not in roles:                        # ¿su rol está en la lista permitida?
            raise HTTPException(403, "Su rol no tiene permiso para esta acción")   # si no: 403
prohibido
        return usuario
    return verificar
```

Cada ventanilla dice qué roles acepta. Por ejemplo, la de farmacia solo acepta `farmacia` y `admin`.
¿Por qué en la cocina y no solo en el menú? Porque ocultar un botón no es seguridad: alguien podría escribir la dirección a mano. La cocina **siempre** revisa.

5.3 `models.py`: los planos de la bodega

Cada **clase** de Python es una **tabla** de la base de datos, y cada línea es una **columna**:

```
class Paciente(Base):
    __tablename__ = "pacientes"                                    # nombre de la tabla
    id: Mapped[uuid.UUID] = mapped_column(Uuid, primary_key=True, default=uuid.uuid4)  # identificador
único
    tipo_documento: Mapped[str] = mapped_column(String(3))         # CC, TI, RC… (tabla RIPS)
    numero_documento: Mapped[str] = mapped_column(String(20), index=True)
    primer_nombre: Mapped[str] = mapped_column(String(60))
    fecha_nacimiento: Mapped[date] = mapped_column(Date)
    sexo: Mapped[str] = mapped_column(String(1))                   # H, M, I (codSexo RIPS)
    cod_municipio_residencia: Mapped[str] = mapped_column(String(5), default="11001")  # DIVIPOLA
    alergias: Mapped[str | None] = mapped_column(Text)             # "| None" = puede quedar vacío
    # Ley 1581 de 2012: autorización expresa para tratar datos sensibles
    autoriza_tratamiento_datos: Mapped[bool] = mapped_column(Boolean, default=False)
```

¿Por qué tantos códigos (DIVIPOLA, codSexo…)? Porque los campos se diseñaron **desde la norma**: los RIPS (Res. 2275 de 2023) exigen esos códigos exactos. Si el dato se guarda bien desde el principio, el RIPS sale bien sin reprocesos.

Las tablas principales:

Tabla	Qué guarda
`usuarios`	Personal: correo, clave cifrada, rol, registro profesional, MFA
`pacientes`	Datos demográficos y de afiliación
`atenciones`	Cada folio de la historia (consulta, urgencia…), su estado (borrador/firmada) y su hash
`diagnosticos`, `procedimientos`, `prescripciones`	Lo que va dentro de cada atención
`ledger_bloques`	La cadena tipo blockchain: cada acción importante
`citas`, `facturas`, `camas`, `hospitalizaciones`, `productos` (farmacia), `ordenes_servicio` (laboratorio)…	Los demás módulos (en `models_ext.py`)

5.4 `schemas.py`: los formularios

Antes de que un dato entre a la cocina, se revisa contra un **formulario** con reglas:

```
class PacienteBase(BaseModel):
    tipo_documento: TipoDoc                        # solo acepta los tipos de la lista oficial
    numero_documento: str = Field(min_length=3, max_length=20, pattern=r"^[A-Za-z0-9]+$")  # solo
letras y números
    primer_nombre: str = Field(min_length=1, max_length=60)
    fecha_nacimiento: date                         # debe ser una fecha válida
    sexo: Literal["H", "M", "I"]                   # solo esas tres letras
    email: EmailStr | None = None                  # si viene, debe parecer un correo
```

Si alguien envía `sexo: "X"` o un documento con símbolos raros, FastAPI lo rechaza **automáticamente**, con un mensaje claro, sin escribir una sola línea más.

¿Por qué separar planos (`models`) de formularios (`schemas`)? Porque lo que se guarda no es igual a lo que se recibe o se muestra. Por ejemplo, la tabla de usuarios guarda la clave **cifrada**, pero ese dato **jamás** se envía a la pantalla: el formulario de salida simplemente no lo incluye.

5.5 `api/`: las ventanillas

Hay un archivo por área: `pacientes.py`, `atenciones.py`, `facturacion.py`, `farmacia.py`, `hospital.py`, `laboratorio.py`, `rips.py`, `rda.py`, etc. Cada función con `@router.post(...)` o `@router.get(...)` es un **endpoint**, un plato de la carta.

Puede ver la carta completa con el sistema corriendo en `http://localhost:8000/docs` (solo desde el servidor): FastAPI la genera sola.

Verbo	Significa	Ejemplo
`GET`	Consultar	`GET /api/pacientes` → lista de pacientes
`POST`	Crear o ejecutar una acción	`POST /api/atenciones/{id}/firmar`
`PUT` / `PATCH`	Modificar	`PUT /api/atenciones/{id}` → guardar borrador
`DELETE`	Eliminar	Solo borradores; lo firmado nunca

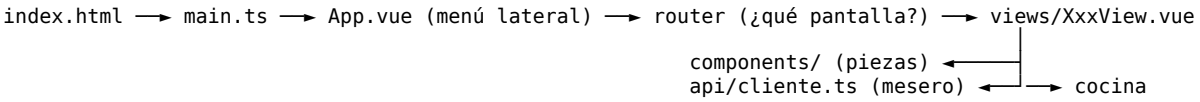
5.6 `services/`: las recetas

Aquí está la **inteligencia** del sistema. Se separó de las ventanillas para que una misma receta se use desde varios lugares y se pueda probar sola.

Archivo	Qué hace
`ledger.py`	La cadena tipo blockchain: agrega bloques y verifica que nadie la haya alterado
`historia.py`	Calcula la huella (hash) de un folio y lo marca como firmado
`rips.py`	Arma el JSON de RIPS según la Res. 2275 de 2023
`facturacion.py`	Facturas, pre-auditoría de glosas
`cie10.py`, `ia.py`	Buscador CIE-10 y sugerencias con IA (local o Claude)
`fhir.py`, `rda.py`	Interoperabilidad: FHIR R4 y el RDA de la Res. 1888 de 2025
`pdf.py`	PDF de la historia y la fórmula, con código QR de verificación
`seguridad_clinica.py`	Alertas: alergias, interacciones, eventos SIVIGILA

Archivo	Qué hace
`bi.py`, `modulos.py`	Constructor de reportes y módulos sin programar
`mfa.py`	Segundo factor (códigos de 6 dígitos)
`anclaje.py`	Publica la huella de la cadena en OpenTimestamps (sello de tiempo externo)

6. El salón (frontend), capa por capa



Archivo	Para qué
`index.html`	La única página. Tiene un espacio vacío (`<div id="app">`) donde Vue dibuja todo
`main.ts`	Arranca Vue, la memoria de sesión y el mapa de pantallas
`App.vue`	El marco: logo, menú lateral filtrado por rol, nombre del usuario y "Cerrar sesión"
`router/index.ts`	El mapa: `/pacientes` → pantalla de pacientes, etc. Si el rol no puede entrar, lo devuelve al inicio. Si la clave es temporal, lo lleva a "Mi cuenta"
`stores/auth.ts`	Recuerda quién está conectado y **cierra la sesión tras 15 minutos sin uso**
`api/cliente.ts`	El mesero: agrega el "pase de entrada" a cada pedido y traduce los errores al español
`views/`	Las 23 pantallas
`components/`	Piezas que se reutilizan: buscador CIE-10, firma con el dedo, dictado por voz, gráficos

6.1 Un archivo `.vue` tiene tres partes

Cada pantalla es un archivo `.vue` con:

```
<script setup lang="ts">
  // 1) LA LÓGICA: qué datos tiene la pantalla y qué pasa al pulsar botones
</script>
```

```
<template>
  <!-- 2) LO QUE SE VE: el HTML con los campos y botones -->
</template>


<style scoped>
  /* 3) EL ESTILO de esa pantalla (opcional) */
</style>
```

6.2 El mesero

```
export const api = axios.create({ baseURL: '/api' })      // todos los pedidos van a /api/...


api.interceptors.request.use((config) => {
  const token = sessionStorage.getItem('token')            // el pase de entrada de la sesión
  if (token) config.headers.Authorization = `Bearer ${token}`   // se adjunta a cada pedido
  return config
})
```

¿Por qué `sessionStorage` y no `localStorage`? Porque `sessionStorage` se borra al cerrar el navegador. En una IPS los computadores se comparten: si alguien olvida cerrar sesión, al cerrar el navegador la sesión muere.

---

7. El viaje completo de un clic: "Firmar la atención"

Siga este recorrido con los archivos abiertos: es la mejor forma de entender cómo encaja todo.

Paso 1. El médico pulsa el botón (`frontend/src/views/AtencionFormView.vue`):

```
async function guardar(firmar = false) {
  if (firmar && !confirm(texto)) return                    // "¿Firmar ahora? El folio quedará
sellado"
  const { data } = atencionId.value
    ? await api.put(`/atenciones/${atencionId.value}`, payload())   // 1º guarda lo último que escribió
    : await api.post('/atenciones', payload())
  if (firmar) {
    await api.post(`/atenciones/${data.id}/firmar`)          // 2º pide firmar
    router.push(`/atenciones/${data.id}`)                    // 3º muestra el folio sellado
  }
}
```

Paso 2. El mesero lo lleva. `api/cliente.ts` adjunta el token y envía `POST /api/atenciones/…/firmar`.

Paso 3. El portero lo deja pasar. `nginx` (`frontend/nginx.conf`) ve que empieza por `/api/` y lo reenvía al contenedor del backend.

Paso 4. La puerta de la cocina. `main.py` le asigna un código de seguimiento.

Paso 5. El guardia revisa. En `deps.py`, `usuario_actual` valida el token (¿es real?, ¿venció?, ¿la cuenta está activa?, ¿debe cambiar su clave?). Luego `requiere_rol` revisa que el rol pueda firmar.

Paso 6. La ventanilla aplica las reglas (`backend/app/api/atenciones.py`, función `firmar`):

```
    if a.estado == "firmada":                raise HTTPException(409, "La atención ya fue firmada")
    if usuario.rol == "estudiante":          raise HTTPException(403, "Los estudiantes no
firman: ...")
    if not a.diagnosticos:                   raise HTTPException(422, "Debe registrar al menos el
diagnóstico principal")
```

Cada `if` es una regla de la norma convertida en código.

Paso 7. Se sella en el libro del notario (`services/ledger.py`):

```
    bloque = BloqueLedger(
        id=(ultimo.id + 1),                                  # número consecutivo
        accion="ATENCION_FIRMADA",
        hash_datos=sha256(json_canonico(datos)),             # huella del contenido del folio
        hash_anterior=ultimo.hash,                           # se ENCADENA al bloque anterior
        ...
    )
    bloque.hash = _hash_bloque(bloque)                       # huella del bloque completo
    bloque.firma = _firmar(bloque.hash)                      # firma con una llave secreta (HMAC)
```

Paso 8. Se marca la atención como firmada (`services/historia.py`): se calcula la huella del folio y se cambia el estado.

```
def marcar_firmada(a):
    a.hash_contenido = calcular_hash(a)       # huella SHA-256 de todo el contenido clínico
    a.estado = "firmada"
    a.firmada_en = datetime.now(timezone.utc)
```

Paso 9. Tareas adicionales: si el diagnóstico es de notificación obligatoria, se registra el evento SIVIGILA. Luego se genera el **RDA** y se pone en cola para la IHCE (Res. 1888 de 2025).

Paso 10. Se guarda todo junto (`db.commit()`). Si cualquier paso falla, **no se guarda nada**: o todo, o nada. Así nunca queda una atención "medio firmada".

Paso 11. La bodega pone su candado. A partir de ahora, si alguien intenta modificar ese folio, **incluso entrando directo a la base de datos sin pasar por la cocina**, PostgreSQL lo rechaza (`core/inmutabilidad.py`):

```
IF OLD.estado = 'firmada' THEN
  RAISE EXCEPTION 'La atención % está firmada y es inalterable (Res. 1995 de 1999)', OLD.id;
END IF;
```

Paso 12. La respuesta vuelve al navegador, y la pantalla muestra el folio con el recuadro "Integridad (sello blockchain)".

---

8. Dónde vive la seguridad en el código

La seguridad está en **capas**, como las puertas de un banco: si una falla, la siguiente detiene el problema.

Capa	Qué protege	Dónde está
1. Contraseñas cifradas con **Argon2id**	Nadie, ni el administrador, puede leer las claves	`core/security.py`
2. **Bloqueo** tras 5 intentos fallidos	Que alguien adivine claves probando	`api/auth.py`
3. **MFA** (código del celular)	Que una clave robada baste para entrar	`services/mfa.py`
4. **Token** con vencimiento y cierre por inactividad	Sesiones abandonadas	`core/security.py`, `stores/auth.ts`

Capa	Qué protege	Dónde está
5. **Roles** revisados en cada ventanilla	Que alguien vea lo que no le corresponde	`core/deps.py` + cada archivo de `api/`
6. **Validación** de formularios	Datos maliciosos o malformados	`schemas.py`
7. **Ledger** encadenado y firmado	Que alguien altere el historial sin que se note	`services/ledger.py`
8. **Triggers** en PostgreSQL	Alterar lo firmado aunque se salten la aplicación	`core/inmutabilidad.py`
9. **Usuario de BD sin privilegios**	Que la aplicación pueda borrar tablas o el ledger	`db/init/01-usuario-aplicacion.sh`
10. **Anclaje externo** (OpenTimestamps)	Que alguien reescriba **toda** la cadena	`services/anclaje.py`
11. **Cabeceras de seguridad** y errores sin detalles	Ataques desde el navegador y filtración de información	`main.py`, `nginx.conf`

¿Cómo detecta el ledger una alteración? Cada bloque guarda la huella del bloque anterior. Si alguien cambia el bloque 50, su huella cambia, el 51 ya no coincide, y la verificación (`verificar_cadena`) lo señala:

```
elif b.hash_anterior != anterior:     problema = "El enlace con el bloque anterior está roto"
elif _hash_bloque(b) != b.hash:       problema = "El contenido del bloque fue modificado"
elif not hmac.compare_digest(...):    problema = "Firma HMAC inválida (bloque recalculado sin la
llave)"
```

Y si alguien recalcula todas las huellas para disimular, le falta la **llave secreta** (`LEDGER_HMAC_KEY`) para firmarlas. Por eso esa llave se guarda fuera del servidor.

---

9. La bodega (base de datos) y sus cambios (migraciones)

Las tablas cambian con el tiempo: hace poco, por ejemplo, se agregó "quién refrendó" a las atenciones. No se puede borrar la bodega y crearla de nuevo, porque se perderían los datos. Por eso existen las **migraciones**: instrucciones numeradas de "remodelación".

Migración	Qué hizo
`0001_esquema_inicial.py`	Creó todas las tablas, los triggers de inmutabilidad y los permisos
`0002_rda_bi_modulos_soporte.py`	Agregó RDA, BI, módulos dinámicos y soporte
`0003_roles_normativos_refrendo_cambio_clave.py`	Agregó el refrendo docente y el cambio de clave obligatorio

Cada vez que arranca, el contenedor `migrate` ejecuta `alembic upgrade head`: "aplica las remodelaciones que falten". Por eso, al hacer `git pull` y `docker compose up`, su base se actualiza sola **sin perder datos**.

---

10. Docker: cómo se empaca y se arranca todo

`docker-compose.yml` describe **cinco** contenedores:

Contenedor	Qué es	Detalle
`db`	PostgreSQL (la bodega)	**No** se expone afuera: solo la cocina puede hablarle
`migrate`	Aplica las migraciones y termina	Por eso aparece como "Exited (0)": es normal
`backend`	La cocina (FastAPI)	Arranca solo después de que `migrate` termine bien
`frontend`	El salón (nginx + pantallas)	El único con puerta hacia usted: el puerto **8080**
`respaldo`	Copia diaria de la base de datos	Y cada domingo prueba que la copia se pueda restaurar

El salón se empaca en dos etapas (`frontend/Dockerfile`):

```
FROM node:22-alpine AS build      # Etapa 1: una "fábrica" con Node que COMPILA las pantallas
RUN npm run build                 #   convierte Vue/TypeScript en archivos que el navegador entiende
FROM nginx:1.27-alpine            # Etapa 2: solo un servidor web liviano con el resultado
COPY --from=build /app/dist /usr/share/nginx/html
```

¿Por qué dos etapas? La fábrica pesa cientos de MB y no se necesita para funcionar. Al final queda solo lo indispensable: menos peso y menos puntos de ataque.

La cocina corre con un usuario sin privilegios (`backend/Dockerfile`):

```
RUN useradd --create-home xentra
USER xentra                       # si alguien lograra entrar al contenedor, no sería "administrador"
```

---

11. Las pruebas automáticas

En `backend/tests/` hay **60 pruebas**: pequeños programas que **usan el sistema como lo haría una persona** y verifican que responda bien. Por ejemplo:

```
def test_login_incorrecto(client):
    r = client.post("/api/auth/login", data={"username": "medico@xentra.co", "password": "malo"})
    assert r.status_code == 401           # con clave mala, DEBE rechazar
```

`assert` significa "asegúrate de que…". Si no se cumple, la prueba falla y avisa.

Las pruebas cubren, entre otras cosas, que:

- un estudiante **no** pueda firmar y el médico sí pueda refrendar;
- el administrador técnico **no** vea historias clínicas;
- una atención firmada **no** se pueda modificar, ni siquiera directo en la base de datos;
- el ledger detecte una alteración;
- el RIPS salga con la estructura de la norma.

¿Para qué sirven? Cada vez que se cambia algo, se corren las 60 en segundos. Si una falla, se sabe de inmediato que el cambio rompió algo, **antes** de que llegue a un paciente. En GitHub, el robot de `.github/workflows/ci.yml` las corre solo en cada cambio.

```
cd backend
pytest -q          # → "60 passed"
```

---

12. Cómo se construyó, en qué orden y por qué

El orden fue deliberado: **de lo más importante y difícil de cambiar, a lo más fácil de cambiar**.

Orden	Qué se hizo	Por qué en ese orden
1	**Leer la norma** (Res. 1995/1999, 2275/2023, 1888/2025, Ley 1581, Ley 2015/2020…)	La norma define qué datos son obligatorios. Si se diseña sin ella, después toca rehacer
2	**Diseñar la bodega** (modelos)	Los datos son lo más difícil de cambiar después. Un mal diseño se arrastra para siempre
3	**Seguridad base** (usuarios, roles, ledger, triggers)	La seguridad se diseña desde el principio. Agregarla al final deja huecos
4	**La cocina** (API y recetas) bloque por bloque, cada uno con sus **pruebas**	Primero que funcione y sea correcto; la pantalla es lo más fácil de cambiar
5	**El salón** (pantallas)	Con la cocina probada, las pantallas solo "piden" y "muestran"
6	**Infraestructura** (Docker, migraciones, respaldos, HTTPS)	Para que funcione igual en cualquier lugar y no se pierdan datos
7	**Documentación** (manuales, protocolos)	Un sistema que nadie sabe operar no sirve

Orden	Qué se hizo	Por qué en ese orden
8	**Mejora continua** (RDA, BI, roles normativos, VPN, logo…)	Cada nueva necesidad se agrega con el mismo método: norma → datos → cocina → pruebas → pantalla → docs

Los bloques de construcción fueron:

1. **Base:** catálogos, sedes, MFA, PDF/QR, plantillas, alertas.
2. **Operación:** agenda, telesalud, facturación, RIPS completo, MUV/DIAN.
3. **Clínico:** urgencias/triage, hospitalización, farmacia, laboratorio, consentimientos.
4. **Interoperabilidad:** FHIR, IA, reportes, anclaje del ledger.
5. **Pantallas** de todos los módulos.
6. **Infraestructura:** Alembic, permisos de BD, HTTPS, respaldos.
7. **Después:** RDA, BI y módulos sin programar, continuidad, roles normativos, protocolos, despliegue.

Principios que se siguieron en todo el código:

- **Nombres en español** (`paciente`, `firmar`, `registrar_evento`): el equipo que lo mantendrá habla español.
- **Comentarios que explican el porqué** y citan la norma, no solo el qué.
- **Una responsabilidad por archivo**: si falla la facturación, se busca en `facturacion.py`.
- **Nunca confiar en la pantalla**: toda regla se revisa en la cocina.
- **Todo o nada** al guardar: nunca datos a medias.
- **Lo firmado no se toca**: se corrige con notas aclaratorias, también selladas.

---

13. Ejercicio: cómo se agregaría un campo nuevo

Supongamos que la IPS quiere guardar la **"EPS secundaria"** del paciente. Estos son los pasos, en orden (es la misma receta para casi cualquier cambio):

Paso	Archivo	Qué se hace
1	`backend/app/models.py`	En la clase `Paciente`, agregar: `eps_secundaria: Mapped[str | None] = mapped_column(String(10))`
2	`backend/alembic/versions/0004_eps_secundaria.py`	Crear la migración que agrega la columna: `op.add_column("pacientes", sa.Column("eps_secundaria", sa.String(10)))`
3	`backend/app/schemas.py`	Agregar `eps_secundaria: str | None = None` en `PacienteBase` (entrada) y

Paso	Archivo	Qué se hace
		en `PacienteOut` (salida)
4	`backend/tests/`	Escribir una prueba: crear un paciente con EPS secundaria y verificar que se guarde
5	`frontend/src/api/tipos.ts`	Agregar el campo al tipo `Paciente`
6	`frontend/src/views/PacientesView.vue`	Agregar el campo en el formulario: `<label>EPS secundaria <input v-model="form.eps_secundaria" /></label>`
7	Terminal	`pytest` (¿todo sigue funcionando?) → `git commit` → `git push`
8	Servidor	`git pull` → `docker compose up -d --build` (el contenedor `migrate` aplica la 0004 solo)

Fíjese en el orden: **bodega → cocina → pruebas → salón → publicar**. Es el mismo orden de la sección 12.

---

14. Qué leer primero si quiere aprender más

Orden sugerido para leer el código (de lo más simple a lo más complejo):

1. `backend/app/core/config.py`: la configuración.
2. `backend/app/models.py`: las tablas `Paciente` y `Atencion`.
3. `backend/app/api/pacientes.py`: una ventanilla sencilla (crear, buscar, ver).
4. `backend/app/services/historia.py` y `ledger.py`: el corazón de la seguridad.
5. `backend/app/api/atenciones.py`: la firma completa (sección 7).
6. `frontend/src/views/PacientesView.vue`: una pantalla sencilla.
7. `frontend/src/views/AtencionFormView.vue`: la pantalla más completa.
8. `backend/tests/test_flujo.py`: el recorrido completo en forma de prueba.

Para aprender cada tecnología (gratis, en español o con subtítulos):

Tema	Dónde
Python desde cero	Documentación oficial en español (`docs.python.org/es`)
FastAPI	Tutorial oficial en español (`fastapi.tiangolo.com/es`)

Tema	Dónde
Vue 3	Guía oficial (`vuejs.org/guide`), con un tutorial interactivo en el navegador
SQL y PostgreSQL	Tutorial oficial (`postgresql.org/docs/current/tutorial.html`)
Docker	"Get started" oficial (`docs.docker.com/get-started`)
Git	Libro *Pro Git*, gratuito y en español (`git-scm.com/book/es`)

Documentos relacionados de este proyecto:

- `docs/manuales/01-MANUAL-DESARROLLO.md`: la versión técnica completa, con todas las normas y decisiones.
- `docs/01-arquitectura.md`: la arquitectura en detalle.
- `docs/04-seguridad-blockchain.md`: el ledger a fondo.
- `docs/05-rips.md` y `docs/06-ia-cie10.md`: RIPS e inteligencia artificial.